

Lab 6 Perform unsupervised learning with clustering

```
In [2]: from sklearn.cluster import KMeans
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from matplotlib import pyplot as plt
%matplotlib inline

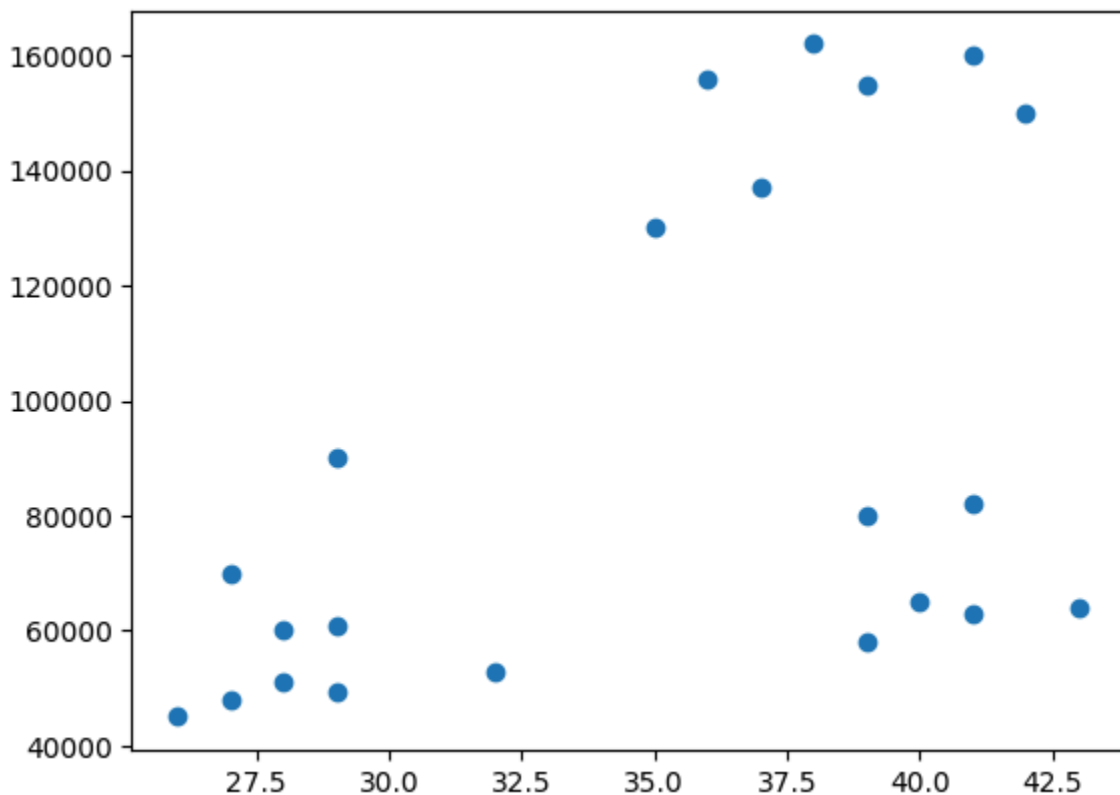
df = pd.read_csv("https://raw.githubusercontent.com/codebasics/py/master/ML/13")
df.head()
```

```
Out[2]:
```

	Name	Age	Income(\$)
0	Rob	27	70000
1	Michael	29	90000
2	Mohan	29	61000
3	Ismail	28	60000
4	Kory	42	150000

```
In [3]: plt.scatter(df['Age'],df['Income($)'])
```

```
Out[3]: <matplotlib.collections.PathCollection at 0x14f32e63a10>
```



```
In [5]: km = KMeans(n_clusters=4)
y_predicted = km.fit_predict(df[['Age', 'Income($)']])
y_predicted
```

```
Out[5]: array([0, 0, 1, 1, 2, 2, 2, 2, 2, 3, 3, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1],
      dtype=int32)
```

```
In [6]: df['cluster'] = y_predicted
df.head()
```

```
Out[6]:
```

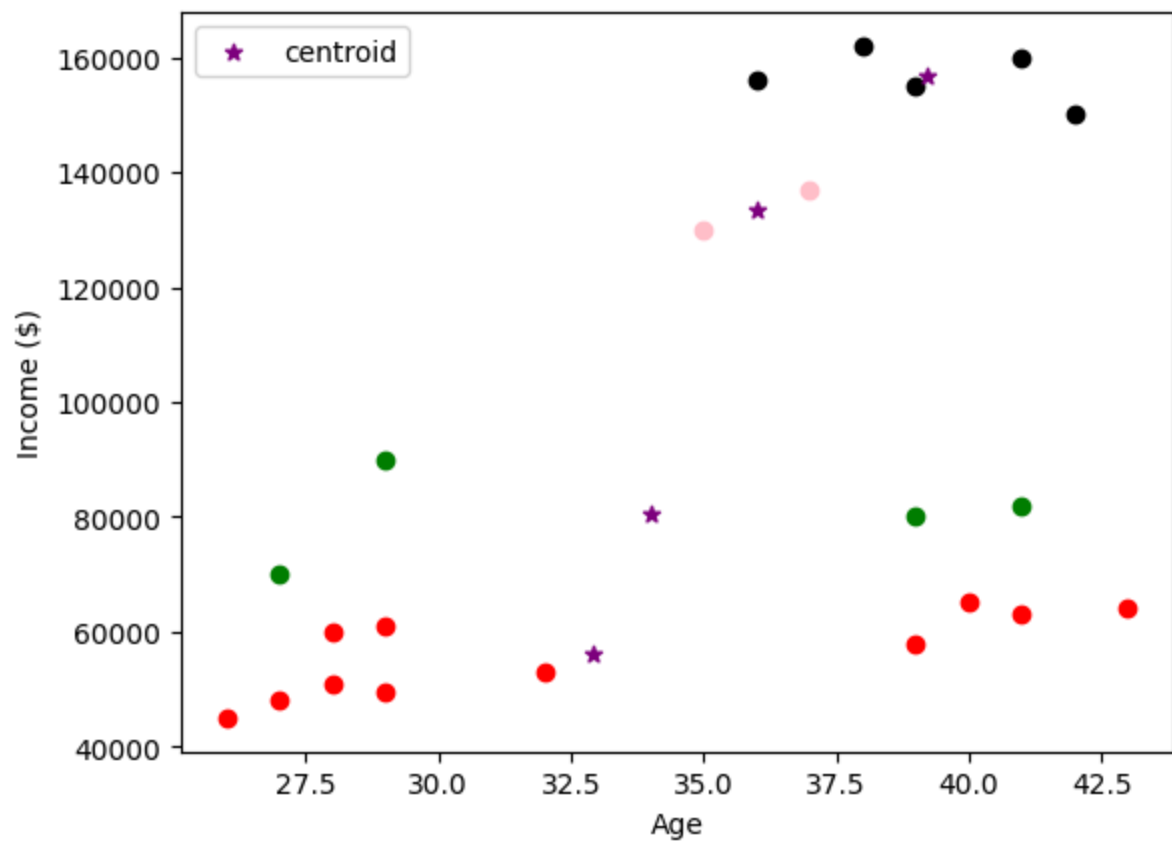
	Name	Age	Income(\$)	cluster
0	Rob	27	70000	0
1	Michael	29	90000	0
2	Mohan	29	61000	1
3	Ismail	28	60000	1
4	Kory	42	150000	2

```
In [7]: km.cluster_centers_
```

```
Out[7]: array([[3.40000000e+01, 8.05000000e+04],
      [3.29090909e+01, 5.61363636e+04],
      [3.92000000e+01, 1.56600000e+05],
      [3.60000000e+01, 1.33500000e+05]])
```

```
In [8]: df1 = df[df.cluster==0]
df2 = df[df.cluster==1]
df3 = df[df.cluster==2]
df4 = df[df.cluster==3]
plt.scatter(df1.Age, df1['Income($)'], color='green')
plt.scatter(df2.Age, df2['Income($)'], color='red')
plt.scatter(df3.Age, df3['Income($)'], color='black')
plt.scatter(df4.Age, df4['Income($)'], color='pink')
plt.scatter(km.cluster_centers_[0], km.cluster_centers_[1], color='purple', marker='x')
plt.xlabel('Age')
plt.ylabel('Income ($)')
plt.legend()
```

```
Out[8]: <matplotlib.legend.Legend at 0x14f32ea41a0>
```



```
In [9]: scaler = MinMaxScaler()

scaler.fit(df[['Income($)']])
df['Income($)'] = scaler.transform(df[['Income($)']])

scaler.fit(df[['Age']])
df['Age'] = scaler.transform(df[['Age']])
df
```

Out[9]:

	Name	Age	Income(\$)	cluster
--	------	-----	------------	---------

0	Rob	0.058824	0.213675	0
1	Michael	0.176471	0.384615	0
2	Mohan	0.176471	0.136752	1
3	Ismail	0.117647	0.128205	1
4	Kory	0.941176	0.897436	2
5	Gautam	0.764706	0.940171	2
6	David	0.882353	0.982906	2
7	Andrea	0.705882	1.000000	2
8	Brad	0.588235	0.948718	2
9	Angelina	0.529412	0.726496	3
10	Donald	0.647059	0.786325	3
11	Tom	0.000000	0.000000	1
12	Arnold	0.058824	0.025641	1
13	Jared	0.117647	0.051282	1
14	Stark	0.176471	0.038462	1
15	Ranbir	0.352941	0.068376	1
16	Dipika	0.823529	0.170940	1
17	Priyanka	0.882353	0.153846	1
18	Nick	1.000000	0.162393	1
19	Alia	0.764706	0.299145	0
20	Sid	0.882353	0.316239	0
21	Abdul	0.764706	0.111111	1

```
In [12]: km = KMeans(n_clusters=4)
y_predicted = km.fit_predict(df[['Age', 'Income($)']])
y_predicted
```

Out[12]: array([1, 1, 1, 1, 3, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0],
dtype=int32)

```
In [13]: df['cluster']=y_predicted
df.head()
```

Out[13]:

	Name	Age	Income(\$)	cluster
0	Rob	0.058824	0.213675	1
1	Michael	0.176471	0.384615	1
2	Mohan	0.176471	0.136752	1
3	Ismail	0.117647	0.128205	1
4	Kory	0.941176	0.897436	3

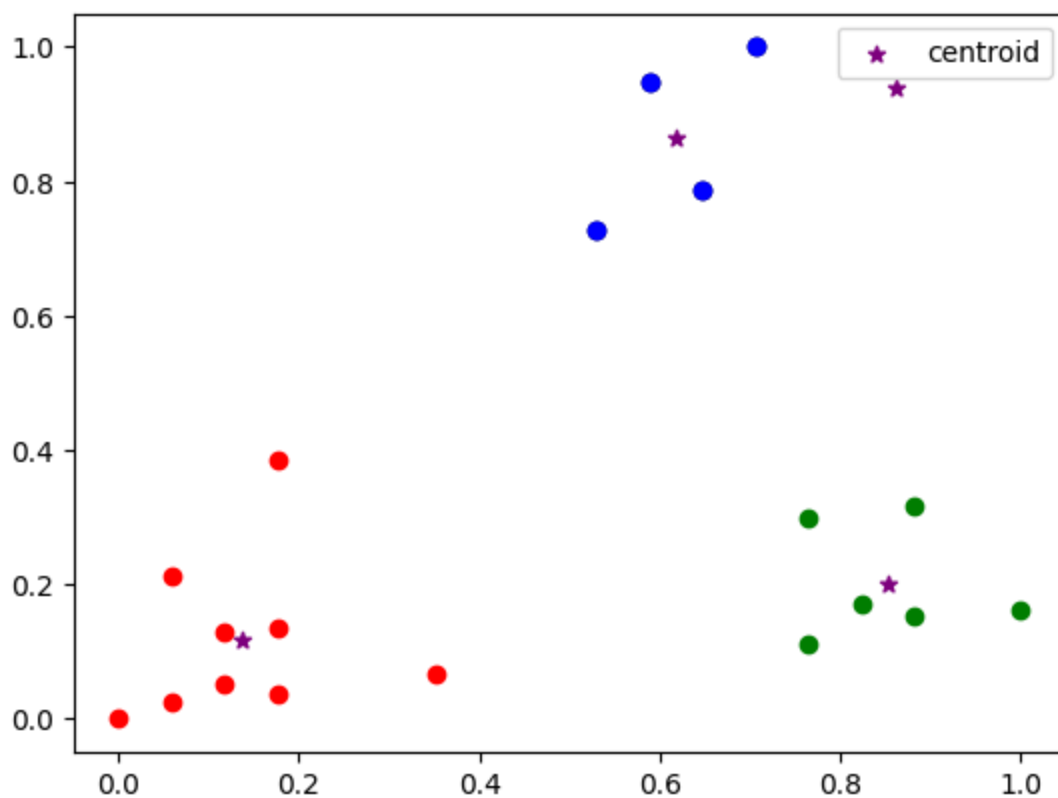
	Name	Age	Income(\$)	cluster
0	Rob	0.058824	0.213675	1
1	Michael	0.176471	0.384615	1
2	Mohan	0.176471	0.136752	1
3	Ismail	0.117647	0.128205	1
4	Kory	0.941176	0.897436	3

In [14]: `km.cluster_centers_`

Out[14]: `array([[0.85294118, 0.2022792],
 [0.1372549 , 0.11633428],
 [0.61764706, 0.86538462],
 [0.8627451 , 0.94017094]])`

In [15]: `df1 = df[df.cluster==0]
df2 = df[df.cluster==1]
df3 = df[df.cluster==2]
plt.scatter(df1.Age,df1['Income($)'],color='green')
plt.scatter(df2.Age,df2['Income($)'],color='red')
plt.scatter(df3.Age,df3['Income($)'],color='black')
plt.scatter(df3.Age,df3['Income($)'],color='Blue')
plt.scatter(km.cluster_centers_[0],km.cluster_centers_[1],color='purple',marker='x')
plt.legend()`

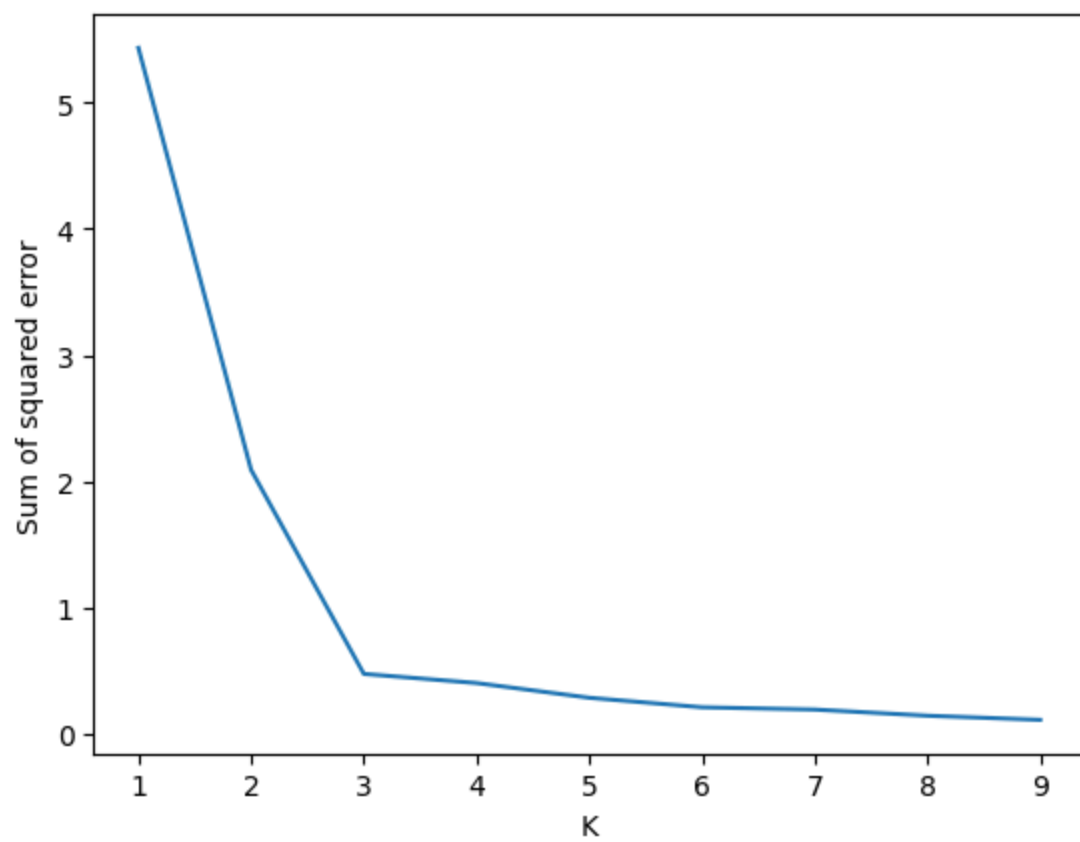
Out[15]: `<matplotlib.legend.Legend at 0x14f38276990>`



```
In [16]: sse = []
k_rng = range(1,10)
for k in k_rng:
    km = KMeans(n_clusters=k)
    km.fit(df[['Age', 'Income($)']])
    sse.append(km.inertia_)

plt.xlabel('K')
plt.ylabel('Sum of squared error')
plt.plot(k_rng, sse)
```

```
Out[16]: [<matplotlib.lines.Line2D at 0x14f382fd810>]
```



In []: